# Brenda Pamela Rodríguez Reyes

3CM31

## I. Fundamentos y recursividad · Ejercicios 1 al 15

### 1. Hola mundo

Se realizó un programa básico en C++ para mostrar un mensaje en pantalla. Se utilizó la función main(), la biblioteca iostream y cout, identificando la estructura de un programa sencillo.

### 2. Suma de dos números con parámetros

Se desarrolló una función que recibe dos números como parámetros, realiza una suma y muestra el resultado. Se practicó el envío de valores a funciones y el uso de variables.

### 3. Calculadora básica

Se creó una calculadora con operaciones de suma, resta, multiplicación y división. La solución en POO agrupa estas operaciones en métodos de una clase, facilitando su organización.

### 4. Calculadora con sobrecarga

Se ampliaron las operaciones usando métodos con el mismo nombre y distintas cantidades de parámetros: cero, dos o tres. Se aplicó sobrecarga para ofrecer distintas maneras de utilizar una operación.

### 5. Calculadora con herencia

Se creó CalculadoraNueva a partir de la calculadora anterior, reutilizando sus operaciones. Se aplicó herencia para aprovechar la funcionalidad de una clase base y extenderla.

### 6. Calculadora con herencia y sobrescritura

Se modificaron operaciones heredadas para calcular multiplicación con sumas sucesivas, potencia con multiplicaciones sucesivas y división con restas sucesivas. Se practicaron herencia y sobrescritura de métodos.

### 7. Creación de tipos de datos

Se definieron datos personalizados para representar un automóvil (precio y año) y una persona (nombre, apellidos, género y edad). Se agruparon sus atributos con estructuras en PE y clases en POO.

### 7 bis. Tamaño de los tipos de datos

Se utilizó sizeof() para consultar cuánta memoria ocupan tipos básicos y personalizados. Se compararon los tamaños de variables, estructuras y clases.

### 8. Promedio de cinco números

Se ingresaron cinco números y se calcularon suma, promedio, máximo y mínimo. Se practicó el procesamiento de valores mediante variables, ciclos y condiciones.

### 9. Promedio con arreglo

Se almacenaron cinco números dentro de un mismo arreglo y se recorrieron mediante índices. Se calcularon suma, promedio, valor máximo y valor mínimo.

### 10. Promedio con puntero al arreglo

Se realizaron las mismas operaciones de cinco números, pero accediendo a sus posiciones de forma indirecta por medio de punteros. Se reforzó la relación entre arreglos y direcciones de memoria.

### 11. Operaciones con matrices

Se trabajó con matrices para realizar multiplicación por una constante y multiplicación entre matrices. Se utilizaron arreglos bidimensionales y ciclos para recorrer filas y columnas.

### 12. Arreglos de datos personalizados

Se formaron arreglos de personas y automóviles a partir de los tipos definidos anteriormente. Se almacenaron varios registros del mismo tipo para consultar sus atributos.

### 13. Arreglos con punteros

Se accedió de manera indirecta a los registros de personas y automóviles mediante punteros. Se practicó la manipulación de arreglos de datos personalizados usando direcciones de memoria.

### 14. Recursividad: factorial, Fibonacci y calculadora

Se aplicaron funciones que se llaman a sí mismas hasta alcanzar un caso base. Se trabajaron factorial, sucesión de Fibonacci y operaciones sucesivas de calculadora expresadas recursivamente.

### 15. Fractales recursivos

Se construyeron patrones como el triángulo de Sierpinski y el polvo de Cantor mediante subdivisiones repetidas. Se observó cómo reglas sencillas y llamadas recursivas producen figuras complejas.

## II. Ordenamientos · Ejercicios 16 al 21

### 16. Ordenamiento burbuja

Se ordenaron números enteros, caracteres y automóviles comparando elementos consecutivos e intercambiándolos cuando estaban fuera de orden. Se programó en PE con funciones y en POO con clases y métodos.

### 17. Ordenamiento Merge Sort

Se dividieron los conjuntos en partes pequeñas, se ordenaron y posteriormente se combinaron. Se aplicó el enfoque de dividir y conquistar para ordenar enteros, caracteres y automóviles, en PE y POO.

### 18. Ordenamiento Quick Sort

Se organizó la información alrededor de un pivote y se ordenaron recursivamente las particiones. Se utilizó el algoritmo con enteros, caracteres y automóviles, tanto con funciones como con clases.

### 19. Burbuja de manera indirecta

Se aplicó burbuja ordenando punteros hacia los datos en lugar de mover los registros originales. Se mostró que es posible visualizar un orden distinto sin modificar el arreglo base.

### 20. Merge Sort de manera indirecta

Se implementó Merge Sort sobre referencias o punteros a los registros. Se ordenaron las referencias, conservando intacta la posición original de los datos.

### 21. Quick Sort de manera indirecta

Se utilizó Quick Sort para reorganizar punteros según la comparación de los valores señalados. La técnica permite consultar los elementos ordenados sin alterar el arreglo original.

## III. Segundo departamental · Tipos de datos y estructuras

### 1. Punteros y autorreferencia

Se crearon estructuras o clases que contienen un puntero a otro elemento de su mismo tipo. Se ejemplificaron relaciones enlazadas entre personas y entre automóviles; en POO se utilizaron clases y encapsulamiento.

### 2. Tipo abstracto de dato (ADT): racional

Se representaron números racionales mediante numerador y denominador, realizando simplificación, suma y multiplicación. En POO se encapsularon los atributos en una clase; en PE se trabajó con estructuras y funciones.

### 3. Pila estática mediante arreglo

Se almacenaron elementos en un arreglo de capacidad fija y se implementaron operaciones para insertar, extraer, mostrar y contar. Se respetó el principio LIFO: el último dato ingresado es el primero que sale.

### 4. Cola estática mediante arreglo

Se organizó una cola en un arreglo de capacidad fija, permitiendo agregar, retirar, mostrar y contar elementos. Se aplicó FIFO y, al extraer, se recorrieron elementos para conservar el orden.

### 5. Lista estática mediante arreglo

Se implementó una lista en un arreglo, con inserción y eliminación por posición, recorrido y conteo. Fue necesario desplazar datos y comprobar que la posición y la capacidad fueran válidas.

### 6. Pila dinámica mediante punteros

Se construyó una pila con nodos enlazados y memoria dinámica, insertando y retirando desde el tope bajo el principio LIFO. Se usaron new y delete; en POO, una clase y un destructor para liberar memoria.

### 7. Cola dinámica mediante punteros

Se utilizaron nodos enlazados para agregar elementos al final y retirarlos desde el frente, siguiendo FIFO. Se practicaron punteros, reserva y liberación de memoria; en POO se encapsuló el comportamiento en una clase.

### 7 bis. Lista dinámica mediante punteros

Se construyó una lista enlazada cuyos nodos contienen datos y referencias al siguiente. Se insertaron y eliminaron elementos en posiciones válidas y se administró la memoria dinámica; en POO se usó una clase con destructor.

### 8. Pila con librería STL (stack)

Se utilizó stack para insertar con push(), retirar con pop() y consultar el tope con top(). Se aprovechó una estructura ya implementada y se organizó su uso mediante métodos de una clase en POO.

### 9. Cola con librería STL (queue)

Se utilizó queue para insertar con push(), retirar con pop() y consultar el primer elemento con front(). Se aplicó FIFO sin programar manualmente los enlaces internos y se trabajó con encapsulamiento en POO.

### 10. Lista con librería STL (list)

Se empleó list para insertar y borrar elementos mediante insert() y erase(), recorrer sus valores y consultar su tamaño. Se utilizaron iteradores y una clase en la versión POO para administrar las operaciones.

## IV. Conclusiones generales de las tres implementaciones

### Arreglos

Las pilas, colas y listas estáticas se construyeron con arreglos de capacidad limitada, utilizando índices y desplazamientos para organizar sus datos. Esta forma permitió comprender las operaciones internas de cada estructura y el control de posiciones.

### Punteros

Las versiones dinámicas se desarrollaron con nodos enlazados mediante punteros. La memoria se reservó y liberó conforme se agregaron o retiraron elementos, comprendiendo el acceso indirecto y la importancia de evitar fugas de memoria.

### Librerías STL

Con stack, queue y list se reutilizaron estructuras proporcionadas por C++, reduciendo el código necesario para insertar, eliminar y consultar datos. Se dio mayor atención al uso correcto de los métodos y a la organización de las clases.